B.TECH 5TH SEMESTER • COMPILER PROJECT REVIEW

Automatic Optimization-Pass Ordering in Compilers

Finding a better sequence of optimization passes than the fixed, one-size-fits-all pipeline



OVERVIEW

What This Project Is About

Compilers improve program performance through optimization passes — Constant Folding, Dead Code Elimination, Loop Optimization, and more.

These passes are applied one after another — and the order they run in can change the final result, because one pass can create new opportunities for another.

GOAL

Automatically find a suitable order of optimization passes for a given program, rather than relying on one fixed sequence.

EXAMPLE — CONSTANT FOLDING

```
int x = 10 * 20;
```

↓ *compiler evaluates at compile time*

```
int x = 200;
```

The output is unchanged, but the generated code does less work at runtime.

PROBLEM STATEMENT

Why a Fixed Order Falls Short

1

One order fits none perfectly

Compilers normally use a fixed, predefined order of optimization passes for every program.

2

Programs differ

Different programs have different characteristics, so the same fixed order may not always give the best result.

3

Passes interact

One optimization pass can create new opportunities for another — order changes what later passes can do.

4

Selection is needed

There is a need to study and automatically select a better order of optimization passes.

BACKGROUND

Common Optimization Passes

Constant Folding	Calculates constant expressions during compilation.
Dead Code Elimination	Removes code that is not needed for the final result.
Common Subexpression Elimination	Avoids repeating the same calculation when possible.
Loop Optimization	Improves the way loops are executed.

BACKGROUND

The Compilation Pipeline



The key insight: changing the order of the optimization-pass stage can change the final result — even though today's compilers run these passes in a fixed pipeline.

BACKGROUND

Why Pass Ordering Matters

Suppose a compiler has three optimization passes:

- **A** Constant Folding
- **B** Dead Code Elimination
- **C** Loop Optimization

A → B → C
Normal / fixed sequence

A → C → B
Alternative order

B → A → C
Alternative order

The project will check different orders and find which order gives better results for the given program.

BACKGROUND

Different Code, Different Best Order

The best order of passes isn't universal — it depends on what the code actually does.

Dead branch behind a constant

```
int a = 5;
if (a > 10) {
  doHeavyWork();
}
```

BEST ORDER

Constant Folding → Dead Code Elim.

Folding a > 10 to false first is what lets Dead Code Elimination see the branch is unreachable and drop it.

Repeated work inside a loop

```
for (i = 0; i < n; i++) {
  x = a * b;
  y = a * b;
}
```

BEST ORDER

Common Subexpr. Elim. → Loop Optim.

Removing the duplicate a * b first gives Loop Optimization a smaller, single computation it can hoist out of the loop.

Unused result after simplification

```
int t = a * b;
int r = 5 + 3;
// t, r unused
```

BEST ORDER

Const. Folding + CSE → Dead Code Elim.

Simplifying first can turn more values into obviously-unused ones, so running Dead Code Elimination last removes more.

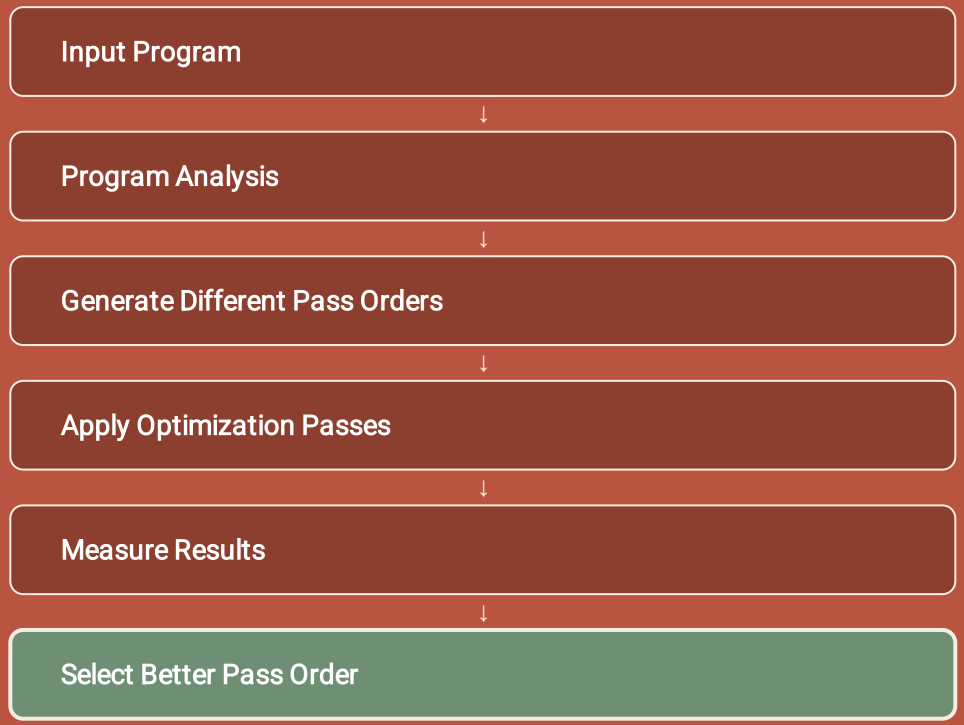
METHODOLOGY

Proposed Methodology — Step by Step

1. **Input**
   Take a C/C++ program as input.
2. **IR Generation**
   Convert the program into an intermediate representation.
3. **Program Analysis**
   Study basic characteristics of the program.
4. **Pass Selection**
   Select a set of compiler optimization passes.
5. **Pass Ordering**
   Try different orders of the selected passes.
6. **Evaluation**
   Measure execution time, code size and compilation time.
7. **Selection**
   Choose the better-performing order.
8. **Comparison**
   Compare it with the normal fixed order.

METHODOLOGY

Proposed System



WHAT MAKES THIS DIFFERENT

Instead of applying passes in one fixed sequence, the system explores multiple orders for the same program and keeps the one that performs best — measured objectively, not assumed.

EVALUATION

Evaluation Metrics

1

Execution Time

Check whether the generated program runs faster.

2

Code Size

Check whether the generated code is smaller.

3

Instruction Count

Compare the amount of generated instructions.

4

Compilation Time

Check the extra time needed to find the pass order.

IMPLEMENTATION

Tools & Technologies

L

LLVM / Clang

Compiler infrastructure and optimization passes.

C

C / C++

Test and benchmark programs.

P

Python

Automating experiments and comparing results.

L

Linux

Development and testing environment.

OUTCOME

Expected Outcome & Next Steps

EXPECTED OUTCOME

Find pass orders that outperform a fixed order for some programs, compared using execution time, code size and compilation time.

Better understanding of how optimization passes affect one another.

NOTE FOR FURTHER RESEARCH

Automatic optimization-pass ordering is already studied in compiler research. After this initial project, existing research papers will be studied to identify a specific improvement or research gap for a final research article or patent work.